

School of Future Tech

Report

on

Inventory Reorder System

C++

Problem Statement 05 | Group 5

by

Subrata Panda

Japji Kaur

Sanika Kangane

Parth Sarova

Cohort: Larry Page

Academic Year 2025–2029

Index

No.	Section	Page
1	Abstract	3
2	Introduction	3
3	Objectives	4
4	Problem Statement	4
5	Core Features	4
6	Methods & Technologies Applied	5
7	Program Implementation	7
8	Source Code	8
9	Output Screenshots	9
10	Time and Space Complexity	10
11	Results and Conclusion	10
12	Future Enhancements and References	11

1. Abstract

The Inventory Reorder System is a Data Structures and Algorithms (DSA) based mini project developed using C++. The project is designed to manage warehouse inventory records efficiently using searching and sorting algorithms. The system allows users to add products, identify low-stock items, sort products by stock and price, search products by ID, and generate reorder lists.

The project implements important DSA algorithms such as Linear Search, Binary Search, Selection Sort, and Quick Sort to demonstrate their practical applications in inventory management systems. Benchmark analysis is also performed to compare the execution time of Linear Search and Binary Search using more than 50 product records.

The project successfully demonstrates how DSA concepts can be applied in real-world warehouse inventory management.

2. Introduction

Inventory management is an important part of warehouse and business operations. Organizations must continuously track product availability, monitor stock levels, and reorder products when inventory becomes low. Manual inventory management can become inefficient and time-consuming, especially when handling large datasets.

The Inventory Reorder System was developed using C++ to solve this problem using Data Structures and Algorithms. The system stores product details such as product ID, name, category, stock quantity, reorder level, and price. Various searching and sorting algorithms are implemented to organize and manage the inventory efficiently.

The project demonstrates the practical implementation of Linear Search, Binary Search, Selection Sort, and Quick Sort algorithms in a real-world warehouse management scenario.

3. Objectives

- To manage warehouse inventory records efficiently.
- To identify low-stock products using Linear Search.
- To sort products using Selection Sort and Quick Sort.
- To search products by ID using Binary Search.
- To compare the performance of Linear Search and Binary Search.
- To generate reorder lists for low-stock products.
- To demonstrate practical applications of DSA concepts using C++.

4. Problem Statement

A warehouse tracks product stock. The system must find low-stock items, sort products by multiple criteria, and benchmark Linear vs Binary Search on 50+ product entries to demonstrate a real performance difference.

5. Core Features

- Add product (ID, name, category, stock, reorder level, price)
- Flag all low-stock products using Linear Search $O(n)$
- Sort products by stock level using Selection Sort
- Sort products by price using Quick Sort
- Search product by ID using Binary Search (after sorting)
- Benchmark Linear vs Binary Search on 50+ products
- Implement at least 2 custom features

6. Methods & Technologies Applied

6.1 Technology Stack

Category	Technology
Programming Language	C++
IDE	Visual Studio Code
Data Structure	Vector
Algorithms Used	Linear Search, Binary Search, Selection Sort, Quick Sort
Timing Library	chrono
Formatting Library	iomanip

6.2 Product Record Management

The project stores all product information using structures and vectors. Each product contains: Product ID, Product Name, Category, Stock Quantity, Reorder Level, and Product Price. Vectors are used because they provide dynamic memory allocation and easy insertion of product records.

6.3 Low-Stock Detection System

The system uses Linear Search to identify products whose stock quantity is less than or equal to the reorder level. The algorithm checks every product one-by-one to detect low-stock items.

Time Complexity	$O(n)$
-----------------	--------

6.4 Stock Sorting System

Selection Sort is used to sort products according to stock quantity. The algorithm repeatedly selects the minimum stock product and places it in sorted order.

Time Complexity	$O(n^2)$
-----------------	----------

6.5 Price Sorting System

Quick Sort is implemented to sort products according to price. Quick Sort improves sorting efficiency for larger datasets.

Time Complexity	$O(n \log n)$
-----------------	---------------

6.6 Product Search System

Binary Search is used to search products by product ID after sorting the dataset. Binary Search repeatedly divides the search space into halves, making searching much faster than Linear Search.

Time Complexity	$O(\log n)$
-----------------	-------------

6.7 Benchmark Analysis System

The chrono library is used to compare the execution time of Linear Search and Binary Search. Benchmark testing is performed using 55 product records. The results show that Binary Search performs faster for larger datasets.

6.8 Custom Features

Reorder List Generator

Displays all products that need restocking.

Category-Based Product Display

Displays products category-wise such as Electronics and Stationery.

7. Program Implementation

The Inventory Reorder System is implemented using a menu-driven approach with modular functions for each operation, improving readability and maintainability.

7.1 Main Functionalities

- Display All Products
- Add Product
- Low Stock Detection
- Sort by Stock
- Sort by Price
- Search Product by ID
- Benchmark Search Algorithms
- Generate Reorder List
- Display Products by Category

7.2 Important Functions Implemented

Function	Purpose
initializeProducts()	Loads default product records into the system
addProduct()	Adds a new product entered by the user
lowStockProducts()	Detects and lists all low-stock items via Linear Search
selectionSortByStock()	Sorts products by stock quantity using Selection Sort
quickSort()	Sorts products by price using Quick Sort
binarySearchByID()	Searches for a product by ID using Binary Search
benchmarkSearch()	Compares execution time of Linear and Binary Search
reorderList()	Generates and displays a reorder list
categoryDisplay()	Shows products grouped by category
menu()	Displays the main interactive menu

8. Source Code

The complete source code is attached separately in the project submission and GitHub repository. Key sections of the implementation are shown below.

Full Source Code — main.cpp

[Screenshot Placeholder: Source Code: Full Source Code — main.cpp]

initializeProducts() Function

[Screenshot Placeholder: Source Code: initializeProducts() Function]

lowStockProducts() — Linear Search Implementation

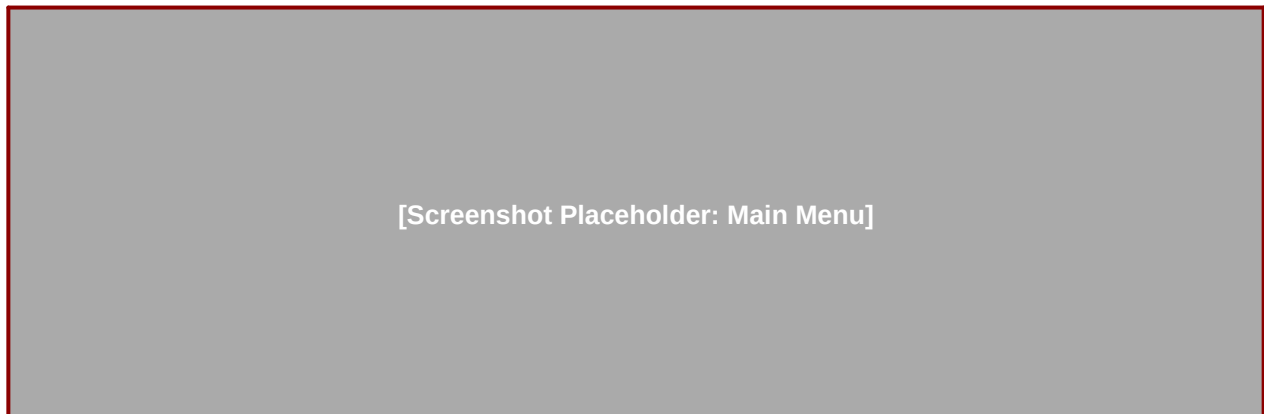
[Screenshot Placeholder: Source Code: lowStockProducts() — Linear Search Implementation]

selectionSortByStock() — Selection Sort Implementation

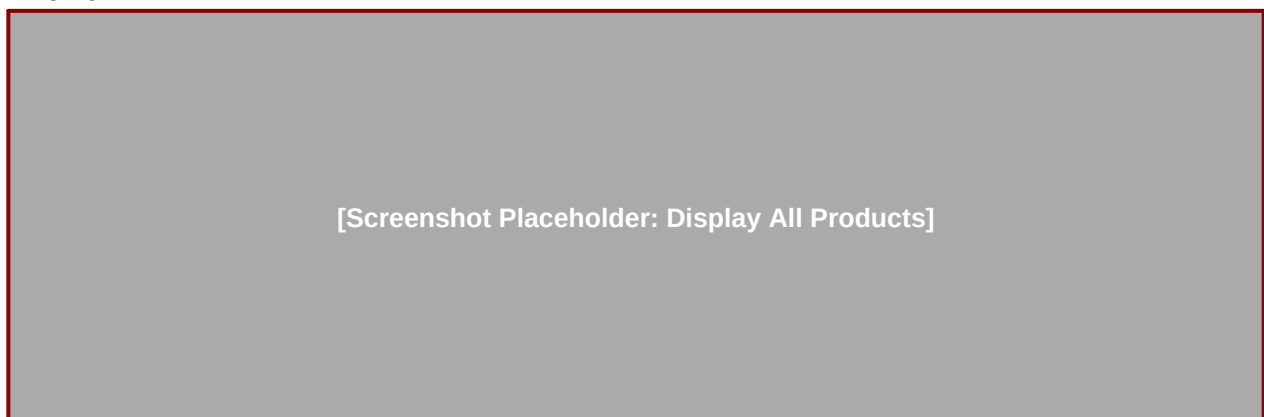
9. Output Screenshots

The following screenshots show the working output of the Inventory Reorder System across all implemented functionalities.

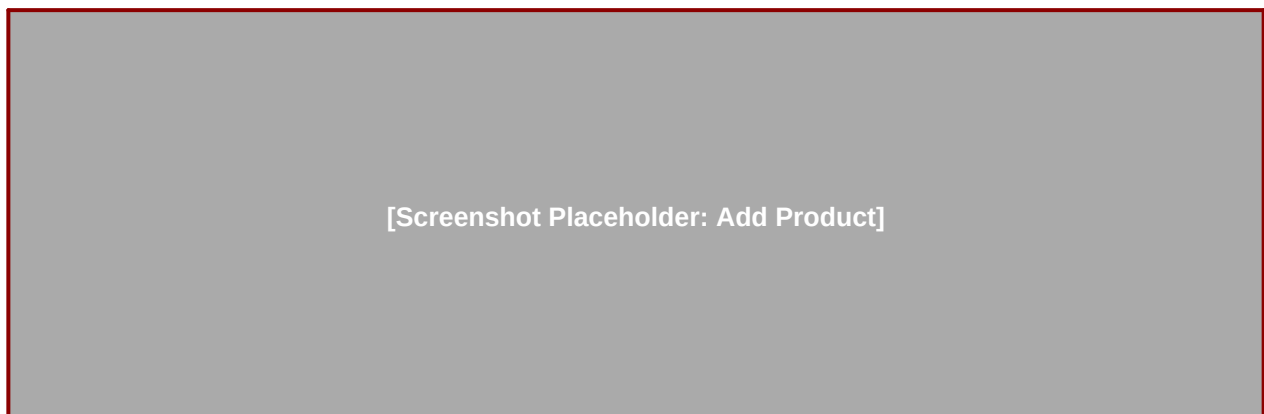
Main Menu



Display All Products



Add Product



Low Stock Products

10. Time and Space Complexity

Algorithm	Purpose	Time Complexity	Space Complexity
Linear Search	Low-stock detection	$O(n)$	$O(1)$
Selection Sort	Sort by stock	$O(n^2)$	$O(1)$
Quick Sort	Sort by price	$O(n \log n)$	$O(\log n)$
Binary Search	Search by ID	$O(\log n)$	$O(1)$

11. Results and Conclusion

Findings

- Product records were managed successfully using structures and vectors.
- Low-stock products were correctly identified using Linear Search.
- Products were sorted accurately by stock quantity using Selection Sort.
- Products were sorted efficiently by price using Quick Sort.
- Binary Search successfully located products by ID after sorting.
- Benchmark testing demonstrated that Binary Search performed faster than Linear Search on datasets containing 50+ products.
- Reorder list generation correctly displayed products requiring restocking.
- Category-wise product display improved inventory organization and readability.
- The menu-driven interface worked correctly for all implemented operations.
- The system successfully handled realistic warehouse product data.

Limitations

- The system is console-based and does not include a graphical user interface.
- Product data is stored temporarily during program execution.
- Database connectivity is not implemented.
- Multi-user inventory management is not supported.
- Benchmark timings may vary depending on system hardware and compiler optimization.
- The project focuses mainly on DSA implementation rather than enterprise-level inventory features.

Conclusion

The Inventory Reorder System successfully implemented important Data Structures and Algorithms concepts including Linear Search, Binary Search, Selection Sort, and Quick Sort in a practical warehouse inventory management application.

The project demonstrated efficient inventory handling, low-stock detection, product sorting, and product searching using more than 50 product records. Benchmark analysis clearly showed the performance

advantage of Binary Search over Linear Search for larger datasets.

The architecture of the system — based on structures, vectors, searching algorithms, sorting algorithms, and menu-driven interaction — proves that real-world inventory operations can be effectively simulated using core DSA concepts in C++. Overall, the project successfully achieved all required objectives while maintaining simplicity, readability, and practical usability for academic demonstration purposes.

12. Future Enhancements and References

Future Enhancements

- Add file handling for permanent data storage.
- Add database connectivity.
- Develop a graphical user interface (GUI).
- Add login and authentication features.
- Implement automatic low-stock notifications.
- Add inventory analytics and report generation.

References

1. GeeksforGeeks — Data Structures and Algorithms references.
2. C++ Documentation — <https://cppreference.com>
3. DSA Lecture Notes — Course material provided by School of Future Tech.
4. Visual Studio Code Documentation — <https://code.visualstudio.com/docs>

Declaration

We hereby declare that this case study report on Inventory Reorder System is our original work. All references and resources used in this report have been properly cited. The application, its code, and the analysis presented in this report are our own work unless otherwise stated.

Subrata Panda
Japji Kaur
Sanika Kangane
Parth Sarova

Group 5 | Problem Statement 05 |
School of Future Tech, ITM Skills University